ms-basic eBNF grammar

February 18, 2026

"MS-Basic" typically refers to **Microsoft BASIC**, the foundational programming language for early personal computers. While there is no single "official" EBNF (Extended Backus-Naur Form) grammar that covers every variant (e.g., Altair BASIC, GW-BASIC, QuickBASIC), the core syntax follows a standard structure.

Core Microsoft BASIC EBNF Grammar

Below is a simplified EBNF representation of the basic structure for classic Microsoft BASIC:

ebnf

```
(* Program Structure *)
program         = { line } ;
line            = line_number statement { ":" statement } newline ;
line_number     = digit { digit } ;

(* Statements *)
statement       = let_stmt | print_stmt | if_stmt | goto_stmt | input_stm
```

Ask anything

```
term            = factor { ( "*" | "/" ) factor } ;
factor          = variable | number | string_literal | "(" expression ")"
variable        = letter [ letter | digit ] [ "$" | "%" | "!" | "#" ] ;
number          = [ "-" ] digit { digit } [ "." { digit } ] ;
string_literal  = '"' { character } '"' ;
```

Use code with caution.

Key Syntax Rules

- **Concatenation:** In Microsoft's EBNF notations, concatenation is often implicit, meaning symbols placed next to each other are treated as a sequence.
- **Line Numbers:** Mandatory in early versions (Altair, GW-BASIC) to define execution order and branching targets.
- **Case Insensitivity:** BASIC keywords are typically case-insensitive, though modern EBNF implementations might use a `~` prefix to denote this explicitly.
- **Type Sigils:** Variable names often end in special characters to denote type: `$` for string, `%` for integer, and `!` or `#` for floating-point. Microsoft Learn +4

For more complex modern implementations like **Visual Basic**, Microsoft uses the ANTLR format for its official language specifications rather than pure EBNF. Microsoft Learn

Syntax Example 1. ::= specifies that registry expression is equivalent to Add Key . Syntax Example 2. ::= | specifies that registr...

Microsoft Learn

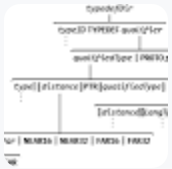
Introduction to Visual Basic

The grammars in this specification are written in ANTLR format -- see http://www.antlr.org/.

Microsoft Learn

Microsoft Macro Assembler BNF Grammar

Microsoft Macro Assembler BNF Grammar * To illustrate the use of the BNF, the following diagram shows the definition of the TYPEDE...



Microsoft Learn

Show all

Please give some examples that use the syntax and show the output of the program.

To illustrate how the EBNF grammar translates into actual code, here are three examples ranging from a simple calculation to a logic-based loop. These follow the classic **GW-BASIC / MS-BASIC** style where line numbers are mandatory.

```
10 PRINT "WHAT IS YOUR NAME?"
20 INPUT NAME$
30 PRINT "HELLO "; NAME$; ", WELCOME TO MS-BASIC!"
40 END
```

Use code with caution.

The Output:

```
WHAT IS YOUR NAME?
? Alice
HELLO Alice, WELCOME TO MS-BASIC!
```

---

2. Mathematical Expression & Let

This demonstrates the `let_stmt` (with the optional `LET` keyword) and the `expression` hierarchy (math order of operations).

The Code:

Use code with caution.

The Output:

```
THE TOTAL IS: 60
```

---

3. Logic and Branching (IF/THEN & GOTO)

This uses the `if_stmt` and `goto_stmt` to create a simple loop and a conditional check. This highlights how the grammar handles program flow via `line_number`.

The Code:

basic

```basic
10 REM COUNTER PROGRAM
20 LET X = 1
30 PRINT "NUMBER:"; X
40 X = X + 1
50 IF X <= 3 THEN GOTO 30
60 PRINT "PROGRAM COMPLETE."
```

```
NUMBER: 2
NUMBER: 3
PROGRAM COMPLETE.
```

Grammar Breakdown of Example 3

To see how the EBNF "sees" the code, look at line `50`:
`50 IF X <= 3 THEN GOTO 30`

Part	EBNF Rule Match
50	`line_number`
IF	`if_stmt` start
X <= 3	`expression` (comparison)
THEN	`if_stmt` separator
GOTO 30	`goto_stmt` (the action)

Would you like me to write a more complex program, such as a **Prime Number Generator**

Since you mentioned being interested in more complex logic, here is a **Number Guessing Game**.

This program utilizes more advanced features of the MS-BASIC grammar, including the `RND` (random number) function, a `GOTO` loop for gameplay, and multiple `IF/THEN` branches for the game logic.

Number Guessing Game (Classic MS-BASIC)

The Code:

basic

```basic
10 PRINT "--- THE GUESSING GAME ---"
20 LET SECRET = INT(RND(1) * 100) + 1
30 LET GUESSES = 0
40 PRINT "I AM THINKING OF A NUMBER BETWEEN 1 AND 100."
50 PRINT "WHAT IS YOUR GUESS";
60 INPUT G
70 LET GUESSES = GUESSES + 1
80 IF G = SECRET THEN GOTO 130
90 IF G < SECRET THEN PRINT "TOO LOW!"
100 IF G > SECRET THEN PRINT "TOO HIGH!"
110 GOTO 50
130 PRINT "CORRECT! IT TOOK YOU"; GUESSES; "TRIES."
```

The Expected Output

```
--- THE GUESSING GAME ---
I AM THINKING OF A NUMBER BETWEEN 1 AND 100.
WHAT IS YOUR GUESS? 50
TOO LOW!
WHAT IS YOUR GUESS? 75
TOO HIGH!
WHAT IS YOUR GUESS? 62
CORRECT! IT TOOK YOU 3 TRIES.
PLAY AGAIN? (1=YES, 0=NO)
? 0
THANKS FOR PLAYING!
```

Grammar Notes on this Example

- **The Semicolon (Line 50):** In the `print_stmt`, a semicolon at the end tells the computer *not* to move to a new line, allowing the `INPUT` prompt to appear on the same line.
- **The RND Function:** While our EBNF simplified this as a `factor`, most MS-BASIC

`expression` and `term` rules.

Would you like to see that Prime Number script, or should we explore how to **convert these rules into a parser** using a tool like Python?

yes

Since you said "yes," let's do both: I'll provide the **Prime Number Finder** script to show nested logic, and then I'll show you how to start a **Python parser** to turn that EBNF grammar into something a computer can actually understand.

---

1. The Prime Number Finder

This script uses a nested loop (a loop inside a loop) to check every number from 2 to 50.

The Code:

```
90 PRINT : PRINT "DONE."
100 END
```

Use code with caution.

The Output:

```
PRIME NUMBERS FROM 2 TO 50:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
DONE.
```

EBNF Insight:
Notice the `:` in line 50. In the EBNF rule `line = line_number statement { ":" statement }`, the colon allows us to chain multiple actions together on a single line number.

---

2. Building a Parser (Python)

To turn your EBNF into a working program, we usually use a **Recursive Descent Parser**.

```
        self.pos = 0

    def parse_statement(self):
        token = self.tokens[self.pos]

        if token == "PRINT":
            return self.parse_print()
        elif token == "LET" or self.is_variable(token):
            return self.parse_let()
        # ... other statements (IF, GOTO)

    def parse_print(self):
        self.pos += 1  # Skip "PRINT"
        content = self.tokens[self.pos]
        self.pos += 1
        return f"Executing: Outputting {content} to screen"

    def parse_let(self):
        # Implementation of: variable "=" expression
        pass

# Example of how the parser would "see" the code
tokens = ["PRINT", "Hello World"]
parser = BasicParser(tokens)
print(parser.parse_statement())
```

Would you like me to expand the **Python Lexer** so you can actually run simple "LET" and "PRINT" commands from a text file?

Please explain the semantics of each kind of BASIC statement

In programming, **syntax** is the structure (the EBNF rules), but **semantics** is the meaning —what actually happens in the computer's memory and CPU when that line is executed.

Here is the semantic breakdown of the core MS-BASIC statements.

---

1. Assignment: `LET`

- **Syntax:** `[LET] variable = expression`
- **Semantics:** The computer evaluates the `expression` on the right side of the equals sign first. Once a single value is calculated, it is stored in the memory location associated with the `variable`. If the variable already held a value, it is overwritten

- **Comma (`,`)**: Moves the cursor to the next "print zone" (usually every 14 columns), creating a tabular look.
- **Semicolon (`;`)**: Keeps the cursor exactly where it is, allowing the next `PRINT` or `INPUT` to start immediately after the last character.
- **Empty PRINT**: Simply sends a "Carriage Return" and "Line Feed," moving the cursor to the start of the next line.

3. Input: `INPUT`

- **Syntax:** `INPUT [prompt_string;] variable`
- **Semantics:** Execution of the program **pauses**. The computer displays a question mark (`?`) on the screen and waits for the user to type something and press Enter. The value typed is then validated (e.g., if the variable is numeric, the user must type a number) and stored in the variable.

4. Branching: `GOTO`

- **Syntax:** `GOTO line_number`
- **Semantics:** This alters the **Program Counter**. Instead of moving to the next chronological line, the interpreter jumps directly to the specified `line_number`. This is the primary way to create infinite loops or return to the start of a logic block.

5. Conditional: `IF ... THEN`

6. Looping: `FOR...NEXT`

- **Syntax:**
  `FOR variable = start TO end [STEP value]`
  `NEXT variable`
- **Semantics:** This is "syntactic sugar" for a complex loop.
  1. The `variable` is set to `start`.
  2. The computer checks if `variable` has passed `end`.
  3. If not, it executes the code until it hits `NEXT`.
  4. At `NEXT`, it adds the `STEP` value (default is 1) to the variable and jumps back to the check in step 2.

7. Documentation: `REM`

- **Syntax:** `REM {character}`
- **Semantics:** "REM" stands for **Remark**. The interpreter ignores everything following this keyword on the current line. It exists solely for the human programmer to explain what the code is doing.

8. Termination: `END`

- **Syntax:** `END`

LET	Memory Storage	Sequential (Next line)
PRINT	Data Output	Sequential
INPUT	Data Acquisition	**Pause**, then Sequential
GOTO	Jump	**Non-sequential** (Jump to target)
IF	Decision	Conditional (Jump or Sequential)
FOR	Iteration	Loop (Repeat block)
END	Termination	**Stop**

Would you like me to explain the **Expression Semantics** (how MS-BASIC handles the order of operations like $2 + 3 * 4$)?

45 sites

Programming Fundamentals: Comprehensive Guide for Beginners

Feb 16, 2026 — Basic syntax in programming is akin to the grammar and punctuation rules used in written language, dictating the structure and com...

Show all

Please output the grammar and the semantics explaination and the examples as plain text.

Here is the complete overview of MS-BASIC including the EBNF grammar, semantic explanations, and code examples in a plain text format.

I. MS-BASIC EBNF GRAMMAR

This grammar represents the structural rules for classic Microsoft BASIC (like GW-BASIC).

```
program = { line }
line = line_number statement { ":" statement } newline
line_number = digit { digit }

statement = let_stmt | print_stmt | if_stmt | goto_stmt | input_stmt | for_stmt | next_stmt |
remark | end_stmt
```

```
expression = term { ( "+" | "-" ) term }
term = factor { ( "*" | "/" ) factor }
factor = variable | number | string_literal | "(" expression ")" | function_call
variable = letter [ letter | digit ] [ "$" | "%" | "!" | "#" ]
number = [ "-" ] digit { digit } [ "." { digit } ]
string_literal = '"' { character } '"'
```

II. SEMANTIC EXPLANATIONS

Semantics describe what the computer actually does when it encounters a specific statement.

1. LET (Assignment)
   Meaning: Evaluates the expression on the right and stores the result in the memory location labeled by the variable on the left. If the variable ends in '$', it stores text; otherwise, it stores a number.
2. PRINT (Output)
   Meaning: Displays the result of expressions to the screen. A semicolon (;) keeps the cursor at the current position, while a comma (,) moves the cursor to the next predefined tab column.
3. INPUT (Data Entry)

6. FOR / NEXT (Definite Iteration)
   Meaning: Creates a loop. The variable is initialized to a starting value. Every time the program hits NEXT, the variable is incremented. If the variable exceeds the 'TO' value, the loop terminates and the program continues after the NEXT statement.
7. REM (Remark)
   Meaning: Provides no instruction to the computer. It is used for internal documentation. The interpreter ignores everything on the line after REM.
8. END (Termination)
   Meaning: Stops the program execution and returns control to the user or operating system.

III. PROGRAM EXAMPLES

EXAMPLE 1: GREETING (Input/Output)
Code:
10 PRINT "ENTER YOUR NAME"
20 INPUT N30$PRINT"HELLO"; N$
40 END

Output:
ENTER YOUR NAME

Output:
GUESS A NUMBER: 10
YOU WIN!

EXAMPLE 3: LOOPING (For/Next)
Code:

```
10 PRINT "COUNTING BY TWOS:"
20 FOR I = 2 TO 10 STEP 2
30 PRINT I;
40 NEXT I
50 PRINT "DONE."
60 END
```

Output:
COUNTING BY TWOS:
2 4 6 8 10 DONE.

Would you like me to create a text file version of this that you can download or save?